

CS6008 — Network Security

Building a Secure Chat Application — Final Report

This report documents the design, implementation, and verification of a chat application hardened across five incremental phases: baseline plaintext relay, Diffie-Hellman key exchange with AES-GCM encryption, PKI-based server authentication, end-to-end encryption between clients, and forward secrecy via automatic key rotation. Each cryptographic phase is accompanied by an attack demonstrating the specific threat it defeats.

Group No: 26

Name : Jay Sonawane

Roll No: 24M0818

Name: Priyanshi Saxena

Roll No: 26M0744

VM Topology

All five phases were implemented and demonstrated across four separate Ubuntu 24.04 LTS virtual machines on a VirtualBox host-only network (192.168.56.0/24), not on a single machine via loopback. Static IP addresses were configured via netplan on each VM to avoid DHCP collisions, and connectivity was verified with ping between every VM pair before any application testing began.

VM	Role	IP Address
Server	Runs the chat server; hosts the Certificate Authority from Phase 3 onward	192.168.56.10
Client1	Chat client (“alice” in testing)	192.168.56.11
Client2	Chat client (“bob” in testing)	192.168.56.12
Mallory	Attacker VM, used for the Phase 2 and Phase 3 MITM tasks	192.168.56.13

Phase 1 — Baseline Chat Application (No Security)

Implementation Summary

A TCP relay server accepting exactly Server (S), Client1 (C1), and Client2 (C2). The server performs no encryption and can read every message it relays — this is the intentional, documented baseline that every later phase measurably improves on.

Framing scheme

TCP is a byte stream with no built-in message boundaries, so every logical message is a single newline-terminated (`\n`) text line. A message is considered “complete” once a newline has been read off the socket; both the client and server buffer partial reads until a full line is available (`LineReader::recv_line()`). This scheme was chosen for simplicity, since all chat content in this assignment is text.

Registration

The first line a client sends after connecting is its username. The server stores connections in a `map<username, socket_fd>` and rejects duplicate usernames with `ERR username_taken`.

Command interface (spec §1.3)

- `@username` message — sends to username, sets it as the current chat partner
- `/chat username` — switches the current chat partner without sending anything
- `/who` — requests the list of online users from the server
- `/quit` — cleanly disconnects

Routing beyond 2 clients

The server's routing is a plain `map<username, socket_fd>` lookup, not hardcoded to exactly two participants. A third, fourth, etc. client connecting with a unique username would route identically via `@username` addressing, with zero protocol changes required — the two-client limit in this phase is a scope constraint from the assignment, not a structural limitation of the code.

Verification

Server-side plaintext logging

The server prints every relayed message to stdout in the format [RELAY] sender -> target: message, directly demonstrating it can read the full content of every message it forwards.

```
jay@server:~$ ./server 5000
[05:26:45] Phase 1 chat server listening on port 5000
[05:26:56] Client connected: alice (192.168.56.11:39884)
[05:27:02] Client connected: bob (192.168.56.12:53542)
[05:27:37] [RELAY] alice -> bob: Hello Bob, this is Alice
[05:27:59] [RELAY] bob -> alice: Hi Alice, how are you?
```

Server log showing full plaintext of every relayed message.

```
jay@client1:~$ ./client 192.168.56.10 5000 alice
Connected as 'alice'. Commands: @user msg | /chat user | /who | /quit
> @bob Hello Bob, this is Alice
>
[ bob ]: Hi Alice, how are you?
> -
```

Client1 (alice) sending and receiving chat messages.

```

jay@client2:~$ ./client 192.168.56.10 5000 bob
Connected as 'bob'. Commands: @user msg | /chat user | /who | /quit
>
[alice]: Hello Bob, this is Alice
> @alice Hi Alice, how are you?
> -

```

Client2 (bob) sending and receiving chat messages.

Wireshark capture

Traffic was captured on the Server VM's enp0s8 interface, filtered to tcp.port == 5000, using tcpdump. Opening the .pcap in Wireshark and using “Follow → TCP Stream” shows the username registration and all chat message content fully readable in plaintext.

```

jay@server:~$ sudo tcpdump -i enp0s8 -w /tmp/phase1_capture.pcap port 5000
[sudo] password for jay:
tcpdump: listening on enp0s8, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C8 packets captured
8 packets received by filter
0 packets dropped by kernel
jay@server:~$ exit
logout
Connection to 192.168.56.10 closed.

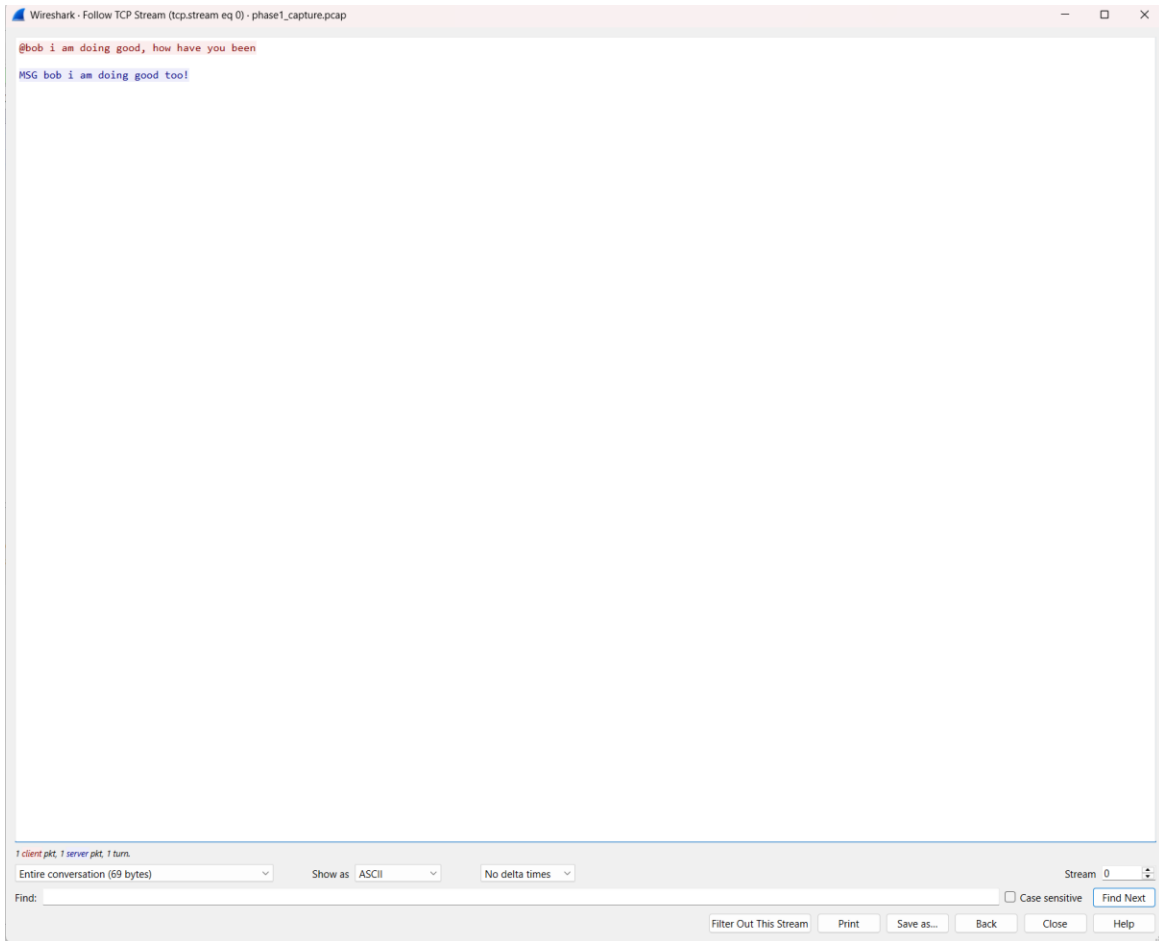
C:\Users\jayso>scp jay@192.168.56.10:/tmp/phase1_capture.pcap "D:\Network Security\A1\"
jay@192.168.56.10's password:
scp: open local "D:\Network Security\A1\"": No such file or directory

C:\Users\jayso>scp jay@192.168.56.10:/tmp/phase1_capture.pcap "D:\Network Security\A1\"
jay@192.168.56.10's password:
phase1_capture.pcap                                100% 822  267.6KB/s   00:00

C:\Users\jayso>

```

tcpdump capture command and packet count on the Server VM.



Wireshark “Follow TCP Stream” showing fully readable plaintext chat content.

Phase 2 — Client-Server Confidentiality via Diffie–Hellman

Implementation Summary

Each client independently establishes an AES-256-GCM session key with the server via a Diffie-Hellman exchange (RFC 3526 MODP Group 14, 2048-bit — verified byte-for-byte against the published RFC text before use). $C1 \leftrightarrow S$ and $C2 \leftrightarrow S$ are two entirely separate exchanges with independent random exponents; there is no shared key between $C1$ and $C2$ at this stage.

Modular exponentiation clarification (per TA guidance)

Functions with “DH” or “Diffie-Hellman” in the name (`DH_generate_key`, `DH_compute_key`, `EVP_PKEY_derive` used for DH, anything from `<openssl/dh.h>`) are disallowed, since they perform the entire exchange automatically. `BN_mod_exp` is a generic bignum primitive, not DH-specific, and is used directly to compute $g^{\text{private}} \bmod p$ for both the public value and the shared secret. The DH protocol logic itself — exponent generation, exchange sequencing, and shared-secret derivation — is implemented from scratch.

Why the raw shared secret is hashed before use as a key

The raw DH output $g^{(ab)} \bmod p$ is not uniformly distributed over the AES-256 key space — it is a number with mathematical structure inherited from modular exponentiation (certain bit patterns are more or less likely, and its distribution is not ideal for a symmetric key). Hashing it with SHA-256 (`key = SHA256(shared_secret)`) collapses it into a uniformly distributed, fixed-length 256-bit value — standard key-derivation practice.

Fixed-length secret encoding

The raw shared secret is converted to bytes via `BN_bn2binpad` with a fixed width equal to the byte-length of the prime, rather than `BN_bn2bin`, which can silently drop leading zero bytes. Without this, the two sides could in rare cases derive different-length “raw secret” byte strings for the same underlying value, breaking the hash-based key derivation.

AES-GCM encryption

Every message after the handshake, including username registration, is encrypted with AES-256-GCM via OpenSSL's EVP API. AES-GCM provides both confidentiality and integrity/authenticity in one primitive — a tampered ciphertext fails the authentication tag check and is rejected, rather than being silently decrypted into corrupted-but-accepted plaintext (unlike plain AES modes such as CBC).

Nonce construction

Since one DH exchange produces a single key used in both directions, nonces are tagged with a 1-byte role marker (`CLIENT_TO_SERVER = 0x01`, `SERVER_TO_CLIENT = 0x02`) followed by an 8-byte counter. This guarantees the two directions can never reuse the same nonce under the same key, even though each side's counter independently starts at 0.

Verification

Fingerprint matching

Both client and server print `SHA256(SHA256(shared_secret))` — i.e., a hash of the derived key, never the raw secret or the key itself — as a “fingerprint,” for the same connection. The values matched exactly across multiple independent connections, and differed between different clients' connections, confirming the exchanges really are independent.

```
jay@server: ~/phase2$ ./server 5000
[06:18:42] Phase 2 chat server (DH + AES-GCM) listening on port 5000
[server<-192.168.56.11:44650] DH handshake complete. Key fingerprint: f350b811b315d76bb59d57ba43316306b
b7348d50d886bd0caddb12e3156d375
[06:18:58] Client connected: alice (192.168.56.11:44650) fingerprint=f350b811b315d76bb59d57ba43316306bb
7348d50d886bd0caddb12e3156d375
[server<-192.168.56.12:51820] DH handshake complete. Key fingerprint: 7aadde23fbb2bdf1876d3a340053699c
87b4f4e1217bfc982779589966ac47f
[06:19:06] Client connected: bob (192.168.56.12:51820) fingerprint=7aadde23fbb2bdf1876d3a340053699c87b
4f4e1217bfc982779589966ac47f
|
```

Server log showing independent DH fingerprints for alice's and bob's connections.

```
jay@client1: ~/phase2$ ./client 192.168.56.10 5000 alice
[client] DH handshake complete. Key fingerprint: f350b811b315d76bb59d57ba43316306bb7348d50d886bd0caddb1
2e3156d375
Verify this fingerprint matches the server's log for your connection: f350b811b315d76bb59d57ba43316306b
b7348d50d886bd0caddb12e3156d375
Connected as 'alice' (encrypted). Commands: @user msg | /chat user | /who | /quit
> |
```

Client1's fingerprint, matching the server's log for alice's connection.

```
jay@client2: ~/phase2$ ./client 192.168.56.10 5000 bob
[client] DH handshake complete. Key fingerprint: 7aadde23fbb2bdf1876d3a340053699c87b4f4e1217bfc9827795
89966ac47f
Verify this fingerprint matches the server's log for your connection: 7aadde23fbb2bdf1876d3a340053699c
87b4f4e1217bfc982779589966ac47f
Connected as 'bob' (encrypted). Commands: @user msg | /chat user | /who | /quit
> |
```

Client2's fingerprint, matching the server's log for bob's connection (a different value from alice's).

```
jay@server: ~/phase2$ ./server 5000
[06:18:42] Phase 2 chat server (DH + AES-GCM) listening on port 5000
[server<-192.168.56.11:44650] DH handshake complete. Key fingerprint: f350b811b315d76bb59d57ba43316306b
b7348d50d886bd0caddb12e3156d375
[06:18:58] Client connected: alice (192.168.56.11:44650) fingerprint=f350b811b315d76bb59d57ba43316306bb
7348d50d886bd0caddb12e3156d375
[server<-192.168.56.12:51820] DH handshake complete. Key fingerprint: 7aadde23fbb2bdf1876d3a340053699c
87b4f4e1217bfc982779589966ac47f
[06:19:06] Client connected: bob (192.168.56.12:51820) fingerprint=7aadde23fbb2bdf1876d3a340053699c87b
4f4e1217bfc982779589966ac47f
[06:22:26] [RELAY] alice -> bob: Hello Bob
[06:22:53] [RELAY] bob -> alice: Hii Alice, how are you?
```

Server log showing the encrypted relay in normal operation.

```
jay@server: ~/phase2 x jay@client1: ~/phase x jay@client2: ~/phase x jay@mallory: ~/phase x + - □ X
jay@client1:~/phase2$ ./client 192.168.56.10 5000 alice
[client] DH handshake complete. Key fingerprint: f350b811b315d76bb59d57ba43316306bb7348d50d886bd0caddb1
2e3156d375
Verify this fingerprint matches the server's log for your connection: f350b811b315d76bb59d57ba43316306b
b7348d50d886bd0caddb12e3156d375
Connected as 'alice' (encrypted). Commands: @user msg | /chat user | /who | /quit
> @bob Hello Bob
>
[alice]: Hii Alice, how are you?
> |
```

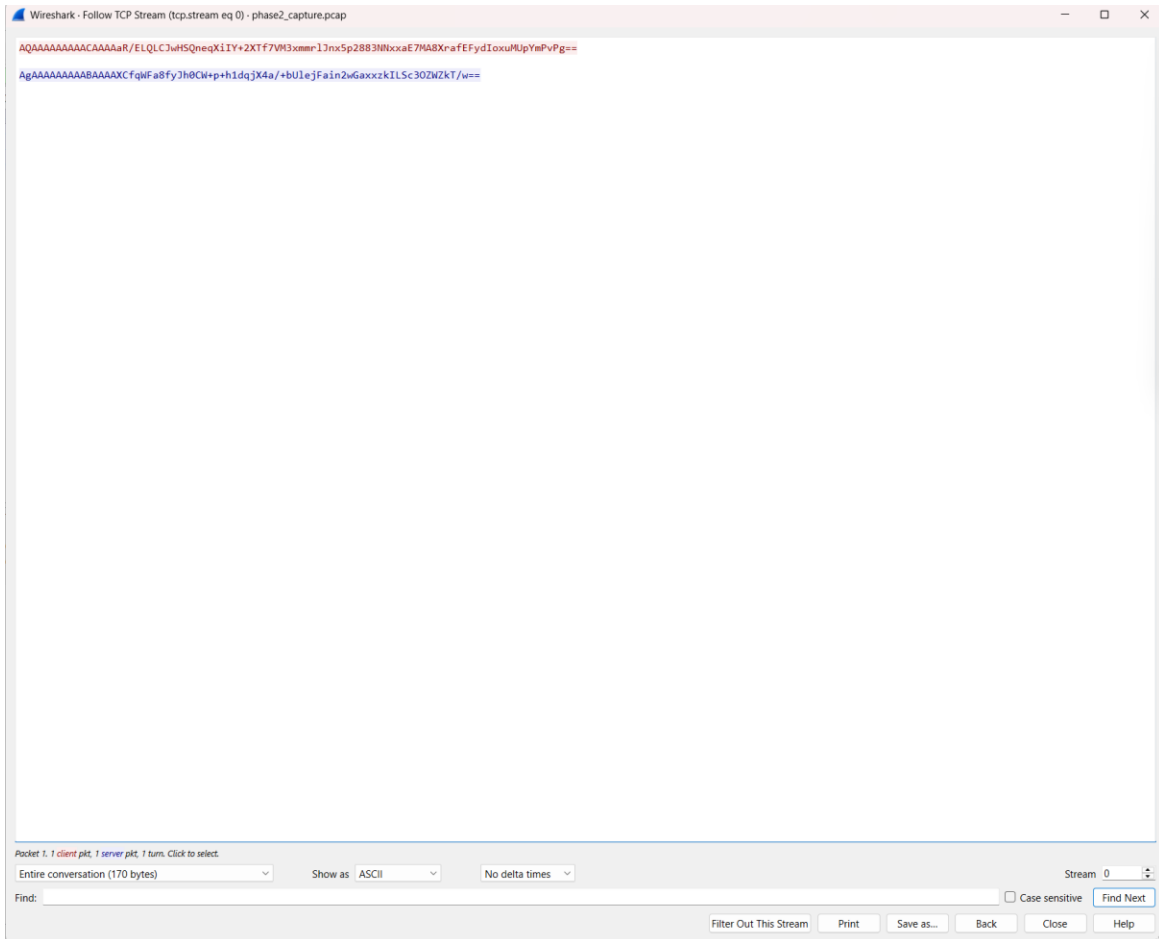
Client1 sending an encrypted message.

```
jay@server: ~/phase2 x jay@client1: ~/phase x jay@client2: ~/phase x jay@mallory: ~/phase x + - □ X
jay@client2:~/phase2$ ./client 192.168.56.10 5000 bob
[client] DH handshake complete. Key fingerprint: 7aadde23fbba2bdf1876d3a340053699c87b4f4e1217bfc9827795
89966ac47f
Verify this fingerprint matches the server's log for your connection: 7aadde23fbba2bdf1876d3a340053699c
87b4f4e1217bfc982779589966ac47f
Connected as 'bob' (encrypted). Commands: @user msg | /chat user | /who | /quit
>
[alice]: Hello Bob
> @alice Hii Alice, how are you?
> |
```

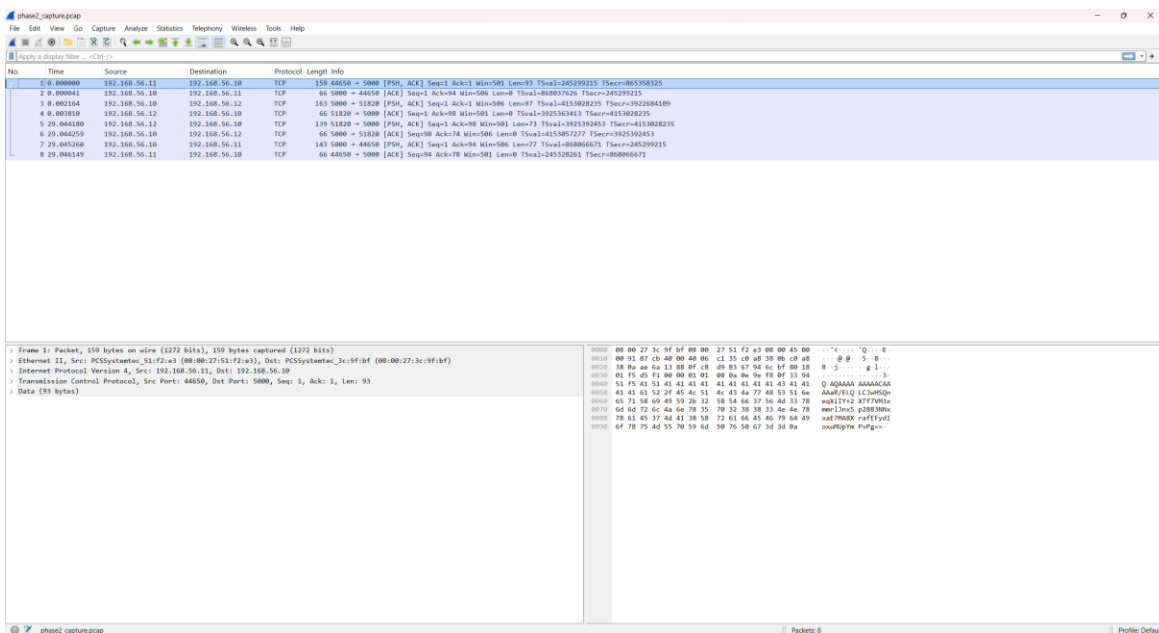
Client2 receiving the encrypted message correctly.

Wireshark re-capture

The identical capture process from Phase 1 was repeated on the same port. This time, “Follow → TCP Stream” shows only base64-encoded ciphertext — no readable usernames or message content, in direct contrast with the Phase 1 capture.



Wireshark stream showing only ciphertext — contrast directly against the Phase 1 plaintext capture.



Packet list for the captured session, showing TCP segments carrying encrypted payloads.

Tamper-detection test

A standalone program (`tamper_test.cpp`) encrypts a sample message, decrypts the unmodified ciphertext successfully, then flips a single bit in the ciphertext and attempts decryption again. The tampered version is rejected with a GCM authentication tag mismatch — no plaintext is produced, demonstrating AES-GCM's tamper-detection property that plain AES modes lack.

MITM Attack (spec §3.3)

What was built

A proxy (`mallory_proxy.cpp`) run on the Mallory VM. The victim client was manually configured to connect to Mallory's IP:port instead of the real server's. Mallory performs two independent DH handshakes — one with the victim (posing as the server), one with the real server (posing as the client) — obtaining two separate session keys. It then decrypts every message from one side, logs the plaintext, re-encrypts under the other side's key, and forwards it, transparently bridging the two connections.

Result

The attack fully succeeded. The victim's client printed a fingerprint that exactly matched what Mallory computed for its victim-facing connection, so from the victim's perspective the handshake looked entirely legitimate. The real server saw a completely different fingerprint for the same username, corresponding to Mallory's server-facing connection. Mallory's log showed the full captured plaintext, including the username and every chat message, while the victim (and the innocent second client) both observed correct message delivery with no visible errors.

```
jay@client1:~/phase2$ ./client 192.168.56.13 5000 alice
[client] DH handshake complete. Key fingerprint: f171651efdc99f9caabf83e59e42c9d9957432551cd9fccc1214d513fe48630b
Verify this fingerprint matches the server's log for your connection: f171651efdc99f9caabf83e59e42c9d9957432551cd9fccc1214d513fe48630b
Connected as 'alice' (encrypted). Commands: @user msg | /chat user | /who | /quit
> @bob this a secret message
>
```

Victim's client (alice, pointed at Mallory) — fingerprint appears to check out normally from her perspective.

```
jay@mallory:~/phase2$ ./mallory_proxy 5000 192.168.56.10 5000
[07:17:33] Mallory MITM proxy listening on port 5000, forwarding to real server 192.168.56.10:5000
[07:17:33] (Point the victim client at THIS machine's IP and port 5000 instead of the real server.)
[07:19:01] Victim connected from 192.168.56.11
[mallory<-victim] DH handshake complete. Key fingerprint: f171651efdc99f9caabf83e59e42c9d9957432551cd9fccc1214d513fe48630b
[mallory->server] DH handshake complete. Key fingerprint: 4d1e12fcad4897d4d52fc382c7049244abbe92a04e45b356d9d84ce47db664ec
[07:19:01] === Two independent DH exchanges established ===
[07:19:01] Victim-facing fingerprint: f171651efdc99f9caabf83e59e42c9d9957432551cd9fccc1214d513fe48630b (this is what the VICTIM sees and believes is 'the server')
[07:19:01] Server-facing fingerprint: 4d1e12fcad4897d4d52fc382c7049244abbe92a04e45b356d9d84ce47db664ec (a completely separate key the REAL server sees)
[07:19:01] Both endpoints now believe they have a direct, secure connection to each other. Neither does
.
[07:19:01] [CAPTURED victim->server] alice
[07:19:01] [CAPTURED server->victim] OK
[07:20:57] [CAPTURED victim->server] @bob this a secret message
```

Mallory's log: two independent fingerprints (victim-facing vs. server-facing) and the captured plaintext, including the username and chat message.

```

jay@server:~/phase2$ ./server 5000
[06:18:42] Phase 2 chat server (DH + AES-GCM) listening on port 5000
[server<-192.168.56.11:44650] DH handshake complete. Key fingerprint: f350b811b315d76bb59d57ba43316306b
b7348d50d886bd0caddb12e3156d375
[06:18:58] Client connected: alice (192.168.56.11:44650) fingerprint=f350b811b315d76bb59d57ba43316306bb
7348d50d886bd0caddb12e3156d375
[server<-192.168.56.12:51820] DH handshake complete. Key fingerprint: 7aadde23fbb2bdf1876d3a340053699c
87b4f4e1217bfc982779589966ac47f
[06:19:06] Client connected: bob (192.168.56.12:51820) fingerprint=7aadde23fbb2bdf1876d3a340053699c87b
4f4e1217bfc982779589966ac47f
[06:22:26] [RELAY] alice -> bob: Hello Bob
[06:22:53] [RELAY] bob -> alice: Hii Alice, how are you?
[07:07:33] [RELAY] alice -> bob: this text is unreadable in transit
[07:08:02] [RELAY] bob -> alice: ciphertext here too
[07:16:36] Client disconnected: alice
[07:16:39] Client disconnected: bob
[server<-192.168.56.12:47624] DH handshake complete. Key fingerprint: c57db09215fef0b5e0bf6d3b92c096a37
e6b680acccc29ab696c502f52fc0efe
[07:18:43] Client connected: bob (192.168.56.12:47624) fingerprint=c57db09215fef0b5e0bf6d3b92c096a37e6b
680acccc29ab696c502f52fc0efe
[server<-192.168.56.13:35568] DH handshake complete. Key fingerprint: 4d1e12fcad4897d4d52fc382c7049244a
bbe92a04e45b356d9d84ce47db664ec
[07:19:01] Client connected: alice (192.168.56.13:35568) fingerprint=4d1e12fcad4897d4d52fc382c7049244ab
be92a04e45b356d9d84ce47db664ec
[07:20:57] [RELAY] alice -> bob: this a secret message

```

The real server's log — shows a different fingerprint for “alice” than what the victim actually saw.

```

jay@client2:~/phase2$ ./client 192.168.56.10 5000 bob
[client] DH handshake complete. Key fingerprint: c57db09215fef0b5e0bf6d3b92c096a37e6b680acccc29ab696c50
2f52fc0efe
Verify this fingerprint matches the server's log for your connection: c57db09215fef0b5e0bf6d3b92c096a37
e6b680acccc29ab696c502f52fc0efe
Connected as 'bob' (encrypted). Commands: @user msg | /chat user | /who | /quit
>
[alice]: this a secret message
> |

```

The innocent second client (bob) receiving the message correctly — the attack is fully transparent.

Why the victim could not detect this from inside the protocol

The fingerprint check the victim performed correctly matched what “the server” (actually Mallory) computed, because Mallory genuinely completed a valid DH exchange with the victim — there is no cryptographic anomaly visible from either endpoint's perspective. Diffie-Hellman only guarantees “both sides agree on a secret with whoever answered this connection”; it provides no way to verify who that is. The only way an ordinary user could catch this is out-of-band verification (e.g., reading the fingerprint aloud over a trusted side channel like a phone call), which essentially nobody does in practice. This gap is exactly what Phase 3's certificates are designed to close.

Phase 3 — Server Authentication via PKI

Implementation Summary

A minimal PKI was set up using OpenSSL's CLI: a self-signed root CA (ca.key/ca.crt), and a server certificate (server.crt) signed by that CA for identity CN=chatserver.local.

CA setup commands (from setup_ca.sh)

```
openssl genrsa -out ca.key 4096
openssl req -x509 -new -key ca.key -sha256 -days 3650 -out ca.crt -subj "/CN=MyChatCA"
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr -subj "/CN=chatserver.local"
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365 -sha256
```

Protocol change

Before any DH exchange, the server sends its certificate. The client validates it against its own local, trusted copy of ca.crt, checking, in order:

- Signature chains to the trusted CA — verified via X509_verify() using the CA's public key
- Validity period — via X509_cmp_current_time() on both notBefore and notAfter
- Identity (CN) matches the expected value — via X509_NAME_get_text_by_NID()

If any check fails, the client aborts immediately, sending nothing further — no nonce, no DH public value, no username.

Proof-of-possession

Even a validly-signed certificate only proves an identity was vouched for by the CA at some point in the past; it does not prove the entity on the current live connection actually controls the matching private key. After certificate validation passes, the client sends a random 16-byte nonce; the server signs it with its private key (EVP_DigestSign, RSA-SHA256); the client verifies the signature using the public key embedded in the already-validated certificate (EVP_DigestVerify). This ties “presents a valid certificate” to “is the real entity on this specific connection, right now.”

Only after both checks succeed does the Phase 2 DH handshake and encrypted chat proceed, unchanged.

Verification

Legitimate flow

Client connects, validates the certificate, completes proof-of-possession, then the DH handshake and encrypted chat proceed exactly as in Phase 2.

```
jay@server: ~/phase3$ ./server 5000
[12:03:20] Phase 3 chat server (PKI + DH + AES-GCM) listening on port 5000
[server<-192.168.56.11:53130] DH handshake complete. Key fingerprint: a056b43ca885303dbea894bf41933a0a6e1220778d51e1aab472646781cd8b44
[12:03:27] Client connected: alice (192.168.56.11:53130) fingerprint=a056b43ca885303dbea894bf41933a0a6e1220778d51e1aab472646781cd8b44 [cert+PoP verified by client before this handshake]
[server<-192.168.56.12:51314] DH handshake complete. Key fingerprint: 638a6078b9e23a5eaf63e5ea2a3385b890675a2003d8686d66ae299f6d69b7aa
[12:03:58] Client connected: bob (192.168.56.12:51314) fingerprint=638a6078b9e23a5eaf63e5ea2a3385b890675a2003d8686d66ae299f6d69b7aa [cert+PoP verified by client before this handshake]
[12:25:57] [RELAY] alice -> bob: Hello Bob!
[12:26:23] [RELAY] bob -> alice: Hii Alice, How are you?
```

Server log: two clients connect, each after their client validated cert + proof-of-possession.

```
jay@client1: ~/phase3$ ./client 192.168.56.10 5000 alice
Certificate validated: CN=chatserver.local, signed by trusted CA, within validity period.
Proof-of-possession verified: server controls the private key matching its certificate.
[client] DH handshake complete. Key fingerprint: a056b43ca885303dbea894bf41933a0a6e1220778d51e1aab472646781cd8b44
Verify this fingerprint matches the server's log for your connection: a056b43ca885303dbea894bf41933a0a6e1220778d51e1aab472646781cd8b44
Connected as 'alice' (authenticated + encrypted). Commands: @user msg | /chat user | /who | /quit
> @bob Hello Bob!
>
[alice]: Hii Alice, How are you?
> |
```

Client1 (alice): certificate validated, proof-of-possession verified, matching DH fingerprint.

```
jay@client2: ~/phase3$ ./client 192.168.56.10 5000 bob
Certificate validated: CN=chatserver.local, signed by trusted CA, within validity period.
Proof-of-possession verified: server controls the private key matching its certificate.
[client] DH handshake complete. Key fingerprint: 638a6078b9e23a5eaf63e5ea2a3385b890675a2003d8686d66ae299f6d69b7aa
Verify this fingerprint matches the server's log for your connection: 638a6078b9e23a5eaf63e5ea2a3385b890675a2003d8686d66ae299f6d69b7aa
Connected as 'bob' (authenticated + encrypted). Commands: @user msg | /chat user | /who | /quit
>
[alice]: Hello Bob!
> @alice Hii Alice, How are you?
> |
```

Client2 (bob): same sequence, independent fingerprint.

MITM re-attempt (spec §4.2)

The identical Phase 2 attack setup was repeated — victim pointed at Mallory. This time, Mallory could only present a self-signed certificate (mallory.crt, generated without CA involvement, using the same CN to be maximally deceptive), since it does not possess the CA's private key (ca.key). The victim's client detected that the certificate's signature does not chain to the trusted CA and aborted immediately, before sending anything further.

```
jay@server: ~/ x jay@client1: ~ x jay@client2: ~ x jay@mallory: ~ x Windows Pow x + v - □ x
jay@client1:~/phase3$ ./client 192.168.56.13 5000 alice
ABORT: certificate validation FAILED -- signature does not chain to the trusted CA
No nonce, no DH public value, no password, and no username were sent. Connection closed immediately.
jay@client1:~/phase3$ |
```

Victim's client: ABORT — certificate validation failed, signature does not chain to the trusted CA. Nothing further sent.

```
jay@server: ~/ x jay@client1: ~ x jay@client2: ~ x jay@mallory: ~ x Windows Pow x + v - □ x
jay@mallory:~/phase3$ ./mallory_proxy_phase3 5000
[12:29:40] Loaded mallory.crt -- a SELF-signed certificate, NOT signed by the real CA. This is the only
certificate Mallory can produce, since it does not have ca.key.
[12:29:40] Mallory (Phase 3 attempt) listening on port 5000. Point the victim client at THIS machine in
stead of the real server.
[12:29:57] Victim connected from 192.168.56.11
[12:29:57] Sent our self-signed certificate to the victim. Waiting to see if they proceed...
[12:29:57] *** VICTIM DISCONNECTED WITHOUT SENDING ANYTHING FURTHER. ***
[12:29:57] This means their client's certificate validation FAILED and it aborted immediately -- exactl
y the spec 4.1 requirement (no nonce, no DH value, no data leaked). Contrast this with Phase 2, where t
he same setup fully succeeded.
[12:29:57] Session ended, ready for next attempt.
|
```

Mallory's log: victim disconnected without sending anything further — the attack fails at the very first message.

This is a direct, explicit contrast with Phase 2, where the identical setup fully succeeded — same Mallory VM, same network position, same “point the victim at the attacker” configuration, but the attack now fails at the very first message.

Stolen certificate without the private key (separate test, spec §4.2)

A distinct scenario was tested — an attacker with a copy of the genuine server.crt but not server.key (simulated via stolen_cert_attacker.cpp, which presents the real certificate but signs proof-of-possession challenges with an unrelated key). Certificate validation passed completely (it is a genuine, correctly-signed certificate), but proof-of-possession failed, since the signature did not verify against the certificate's actual embedded public key.

```
jay@server: ~/ x jay@client1: ~ x jay@client2: ~ x jay@mallory: ~ x Windows Pow x + v - □ x
jay@client1:~/phase3$ ./client 192.168.56.13 5000 alice
Certificate validated: CN=chatserver.local, signed by trusted CA, within validity period.
ABORT: proof-of-possession FAILED -- the signature does not verify against the certificate's public key
. This server (or attacker) holds the certificate file but NOT the matching private key. Refusing to pr
oceed.
jay@client1:~/phase3$ |
```

Client: certificate validated successfully, immediately followed by ABORT — proof-of-possession failed.

```
jay@server: ~/ X jay@client1: ~, X jay@client2: ~ X jay@mallory: X Windows Pow X + v - □ X
jay@mallory:~/phase3$ ./stolen_cert_attacker 5000
[12:44:55] Loaded a COPY of the genuine server.crt (this will pass certificate validation) and a WRONG
private key that does NOT match it (this will fail proof-of-possession).
[12:44:55] Stolen-cert attacker listening on port 5000. Point a client at THIS machine to run the test.
[12:45:01] Client connected -- presenting the stolen (but genuine) certificate.
[12:45:01] Client accepted the certificate and sent a proof-of-possession challenge, as expected (the c
ert itself really is valid).
[12:45:01] Sent a signature produced with the WRONG private key. The client should now detect that this
signature does not verify against the certificate's actual public key, and abort.
[12:45:01] *** CLIENT DISCONNECTED after the bad signature. Proof-of-possession correctly rejected the
stolen cert + wrong key combination. ***
[12:45:01] Session ended.
```

Attacker's log: presented the genuine certificate, then signed the challenge with the wrong key.

This demonstrates the two defenses catch two genuinely different attack patterns: forging your own certificate (caught by chain validation) and stealing/relaying someone else's real certificate without their private key (caught by proof-of-possession).

Note on PKI's fundamental limits (discussion point)

Phase 3 defends against an attacker who lacks the CA's private key, exactly as specified. It does not defend against a compromised or careless CA — if an attacker somehow obtained a certificate genuinely signed by the CA for the server's identity, every check described above would legitimately pass. This is an inherent property of PKI systems generally (including the one securing HTTPS across the web), not a gap specific to this implementation; real-world systems add certificate revocation and Certificate Transparency logs to mitigate this, which are outside this assignment's scope.

Phase 4 — End-to-End Encryption Between Clients

Implementation Summary

C1 and C2 establish a shared key directly with each other, independent of and never derivable by the server, triggered by the exact command `/e2e username`. The server's routing logic requires zero changes — `phase4/server.cpp` is byte-for-byte identical to Phase 3's server.

Wire tags (spec §1.4, exact strings as required)

- `__E2E_INIT__` <hex DH public value> — initiates (or re-initiates) the exchange
- `__E2E_ACK__` <hex DH public value> — completes the exchange
- `__E2E_MSG__` <base64 blob> — the actual encrypted chat content

These tags are just the payload of an ordinary `@username`-addressed message sent through the existing, already-encrypted client-server channel from Phase 3. The server's relay code (look up target, forward the payload) never inspects or understands the tag — it is opaque data as far as the server is concerned.

Layering

An E2E chat message is `AES-GCM_e2e(plaintext)`, tagged with `__E2E_MSG__`, and that whole tagged string is then encrypted again via the existing Phase 3 channel: `send_encrypted()` call before hitting the socket — two independent AEAD layers under two independent keys, an additional inner layer rather than a replacement of the earlier links.

Nonce safety for the E2E layer

The same problem from Phase 2 applies here — one DH exchange produces a single key used by both directions. Since both usernames are known to both sides, each side deterministically assigns itself a direction byte by comparing usernames (lexicographically smaller = 0x01, larger = 0x02), guaranteeing the two directions can never collide on a nonce.

Protocol-level distinction between handshake and chat

The receiving client checks the payload's prefix before doing anything else — `__E2E_INIT__` / `__E2E_ACK__` are fed into the DH state machine and never displayed as chat; `__E2E_MSG__` is decrypted and displayed, never treated as a handshake step.

Verification

Fingerprint matching, independent of the server

Both clients print an E2E session fingerprint immediately after the exchange completes. The values matched exactly, and are computed entirely between C1 and C2 — the server never participates in or sees this computation.


```
jay@server: ~/ | jay@client1: ~ | jay@client2: ~ | jay@mallory: ~ | Windows Pow | + | - | □ | × |
jay@client1:~/phase4$ ./client 192.168.56.10 5000 alice
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: b63083caf8cd89dfef11ea7bac1cf19bada3841127c030cd779480
103f2adb2f
Client-server fingerprint: b63083caf8cd89dfef11ea7bac1cf19bada3841127c030cd779480103f2adb2f
Connected as 'alice'. Commands: @user msg | /chat user | /e2e user | /who | /quit
> @bob before E2E
> /e2e bob
[E2E] Initiated key exchange with bob
>
[E2E] Session acknowledged with bob. Fingerprint: 9a8c0432853caf093015ea9a67ee5e75e50cd66ecbdf10765033a
df6bce65312
> |
```

Client1 (alice): E2E session established with bob, fingerprint shown.

```
jay@server: ~/ | jay@client1: ~ | jay@client2: ~ | jay@mallory: ~ | Windows Pow | + | - | □ | × |
jay@client2:~/phase4$ ./client 192.168.56.10 5000 bob
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: 876576d86a9fd7e8e30f11ddd4158065dda6b43cf2408a83c6abca
1e050aa302
Client-server fingerprint: 876576d86a9fd7e8e30f11ddd4158065dda6b43cf2408a83c6abca1e050aa302
Connected as 'bob'. Commands: @user msg | /chat user | /e2e user | /who | /quit
>
[alice]: before E2E
>
[E2E] Session established with alice. Fingerprint: 9a8c0432853caf093015ea9a67ee5e75e50cd66ecbdf10765033
adf6bce65312
> |
```

Client2 (bob): matching E2E fingerprint.

Server-blindness, before vs. after

A message was sent before /e2e (server could read it in full), then /e2e was triggered, then another message was sent. The server's log shows the plaintext message, the __E2E_INIT__/_E2E_ACK__ handshake values (public, not secret), and then the opaque __E2E_MSG__ ciphertext for all subsequent chat — both states visible together in one log.

```
jay@server: ~/ | jay@client1: ~ | jay@client2: ~ | jay@mallory: ~ | Windows Pow | + | - | □ | × |
jay@server:~/phase4$ ./server 5000
[06:22:55] Phase 3 chat server (PKI + DH + AES-GCM) listening on port 5000
[server<-192.168.56.11:55804] DH handshake complete. Key fingerprint: b63083caf8cd89dfef11ea7bac1cf19ba
da3841127c030cd779480103f2adb2f
[06:23:07] Client connected: alice (192.168.56.11:55804) fingerprint=b63083caf8cd89dfef11ea7bac1cf19ba
da3841127c030cd779480103f2adb2f [cert+PoP verified by client before this handshake]
[server<-192.168.56.12:37382] DH handshake complete. Key fingerprint: 876576d86a9fd7e8e30f11ddd4158065d
da6b43cf2408a83c6abca1e050aa302
[06:23:20] Client connected: bob (192.168.56.12:37382) fingerprint=876576d86a9fd7e8e30f11ddd4158065d
da6b43cf2408a83c6abca1e050aa302 [cert+PoP verified by client before this handshake]
[06:24:57] [RELAY] alice -> bob: before E2E
[06:25:40] [RELAY] alice -> bob: __E2E_INIT__A2F63229557A35CB0E32EBD69122056A21EB058E823720BDDFE8C687B9
8F6F59AE1F8CC829CFE88EF4D04DDE88A1D132106CF01204DBB76F77E460A6C4B0F59DB9FB4308EF6552A0AC36EC2B634B897A
546A99DBE3D2B11CEB4933EE8CF900BBC72B6097A9AD5DA1FFEB6BF44BF0F8A11043FA30676A2F191CC96E9A4603B7E4266415A
077CD0C29DE4C615DD48B66A65438B3000CEBF69FD6194B9BA087ED6165DEB83A2AC994860B0205C83477ACB4744D67FC14C075
DE3BF04E8A3B7E97323337A083308C04846C51E61AA84C24E1D0FBDA897C40B9948D0EBF858806D022B8E1249B9E9A8693FF41
5685DD616F979804D29CACB0D5FEBE24AB94331D23
[06:25:40] [RELAY] bob -> alice: __E2E_ACK__30275FC538C347EBC0DA732D966D969B0985F03DA07049E7D87D504080E
990C59F6B85D4830BC5073CD44DFA5973542C6B8D4C68B801338461B369F4B924ECD14DC96FE1A02BF9A0660937E660F94508FB
3254A086D4CD021012B808AE8B56D3B6BE76B0C411A459D6FB84CFB361C5D1386DFF8523F40A1E0F65B96556FAA68BF92CAD37
9B9869BB8967012A2867713DD1543496C9172A8346A2787666C87CA491AC5809FBB6DF44781BB1607C95505EE93666FE67DFEA
1865686D44495D9E7FB1E85BBF78B3D92C3BA24DD4CD8C05F52E9E4F4089E51503AF6F1A6628F4C2CAEC35563FD8D4DE927DFF6
3670769445B8ECCF77BA514498E5EA7CED8F32C28
[06:28:42] [RELAY] alice -> bob: __E2E_MSG__AQAAAAAAAAAAAAAa9SSTQoFDLxEvWja8uJ61HVI4d2NNmyuw==
|
```

Server log: plaintext before /e2e, then opaque ciphertext after — both states in one view.

Functional correctness

The receiving client correctly displayed the decrypted message.

```
jay@server: ~/ x jay@client1: ~ x jay@client2: ~ x jay@mallory: ~ x Windows Pow x + - □ x
jay@client1:~/phase4$ ./client 192.168.56.10 5000 alice
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: b63083caf8cd89dfef11ea7bac1cf19bada3841127c030cd779480
103f2adb2f
Client-server fingerprint: b63083caf8cd89dfef11ea7bac1cf19bada3841127c030cd779480103f2adb2f
Connected as 'alice'. Commands: @user msg | /chat user | /e2e user | /who | /quit
> @bob before E2E
> /e2e bob
[E2E] Initiated key exchange with bob
>
[E2E] Session acknowledged with bob. Fingerprint: 9a8c0432853caf093015ea9a67ee5e75e50cd66ecbdf10765033a
df6bce65312
> after E2E
> |
```

Client1: E2E message sent and received correctly.

```
jay@server: ~/ x jay@client1: ~ x jay@client2: ~ x jay@mallory: ~ x Windows Pow x + - □ x
jay@client2:~/phase4$ ./client 192.168.56.10 5000 bob
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: 876576d86a9fd7e8e30f11ddd4158065dda6b43cf2408a83c6abca
1e050aa302
Client-server fingerprint: 876576d86a9fd7e8e30f11ddd4158065dda6b43cf2408a83c6abca1e050aa302
Connected as 'bob'. Commands: @user msg | /chat user | /e2e user | /who | /quit
>
[alice]: before E2E
>
[E2E] Session established with alice. Fingerprint: 9a8c0432853caf093015ea9a67ee5e75e50cd66ecbdf10765033
adf6bce65312
>
[alice (E2E)]: after E2E
> |
```

Client2: E2E message received and decrypted correctly, tagged (E2E).

Phase 5 — Forward Secrecy

Implementation Summary

The Phase 4 E2E session key is automatically renegotiated every 60 seconds, using an epoch-tagged version of the same `__E2E_INIT__`/`__E2E_ACK__` handshake. `phase5/server.cpp` again requires zero changes.

Collision-avoidance design decision (spec §6.1, explicitly graded)

Both clients' 60-second timers will never fire in perfect sync. The chosen strategy is asymmetric roles: whichever username is lexicographically smaller always initiates rekeys (on its timer, or via a manual `/rekey` test command); the other peer never initiates on its own timer, only ever responds with `__E2E_ACK__`. Since there is structurally only ever one initiator for a given pair, there is no race to resolve — both sides independently compute the identical answer to “who initiates” simply by comparing the two, already-known usernames.

This was chosen over an alternative epoch/timestamp “highest proposal wins” scheme because it avoids the added complexity of detecting and resolving simultaneous proposals entirely, and is sufficient given the usernames are fixed for the session's lifetime.

Epoch tagging so in-flight messages survive a rotation

Every wire message now carries an epoch number: `__E2E_INIT__` `<epoch>`:`<hex pub>`, `__E2E_ACK__` `<epoch>`:`<hex pub>`, `__E2E_MSG__` `<epoch>`:`<base64 blob>`. Each session retains its current key/epoch plus the one previous key/epoch. A message tagged with the previous epoch (e.g., one that was in flight right as a rotation completed) can still be decrypted; anything older than that has already been actively destroyed.

Active key destruction

`OPENSSL_cleanse()` zeroes the old key's memory before it is dropped, once a new key is confirmed — “discarded” is implemented as active erasure, not merely dereferencing the old key and letting it become unreachable.

Additional documented command

`/rekey username` forces an immediate rotation for testing purposes, rather than waiting the full 60 seconds, per the spec's explicit allowance for extra Phase 5 commands (§1.3). It only succeeds for the lexicographically-smaller username in a pair, mirroring the automatic timer's own rule.

Verification

Fingerprint + timestamp across multiple rotations

Both clients log a timestamped fingerprint at every rotation. Across two consecutive rotations (epoch $0 \rightarrow 1 \rightarrow 2$), the fingerprint changed each time, and both clients agreed on the identical value after each rotation completed.

```
jay@server: ~/ X jay@client1: ~ X jay@client2: ~ X jay@mallory: ~ X Windows Pow X + - □ X
jay@client1:~/phase5$ ./client 192.168.56.10 5000 alice
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: 1dd6d72269acc49b36204fc4cc97391b9118b9660dff27f4d78df6
d476c624e7
Client-server fingerprint: 1dd6d72269acc49b36204fc4cc97391b9118b9660dff27f4d78df6d476c624e7
Connected as 'alice'. Commands: @user msg | /chat user | /e2e user | /rekey user | /who | /quit
(/rekey forces an immediate rotation for testing, instead of waiting 60s -- only works if your username
sorts before your peer's, since that side is the designated initiator.)
> /e2e
No chat partner selected.
> /e2e bob
[E2E] Initiated key exchange with bob
>
[13:23:18] [E2E] Session with bob now at epoch 0. Fingerprint: 73bbde310dbe964014783a0e325e594b118114ec
811c01ee62b3142852a13045
> /rekey bob
[REKEY] Forced immediate rotation to epoch 1 with bob (test command)
>
[13:23:28] [E2E] Session with bob now at epoch 1. Fingerprint: 25baaa9cbc618e6e97daea5abb7a72e30d9302ac
dde221878c1404aa82e34371
> |
```

Client1: E2E session established (epoch 0), then rotated to epoch 1 via /rekey — new fingerprint, timestamped.

```
jay@server: ~/ X jay@client1: ~ X jay@client2: ~ X jay@mallory: ~ X Windows Pow X + - □ X
jay@client2:~/phase5$ ./client 192.168.56.10 5000 bob
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: a56c9ca9bdd8e22dc2a028af7e693c81c5ce4b2171db62a77f928d
55f59ae69e
Client-server fingerprint: a56c9ca9bdd8e22dc2a028af7e693c81c5ce4b2171db62a77f928d55f59ae69e
Connected as 'bob'. Commands: @user msg | /chat user | /e2e user | /rekey user | /who | /quit
(/rekey forces an immediate rotation for testing, instead of waiting 60s -- only works if your username
sorts before your peer's, since that side is the designated initiator.)
>
[13:23:18] [E2E] Session with alice now at epoch 0. Fingerprint: 73bbde310dbe964014783a0e325e594b118114
ec811c01ee62b3142852a13045
>
[13:23:28] [E2E] Session with alice now at epoch 1. Fingerprint: 25baaa9cbc618e6e97daea5abb7a72e30d9302
acdde221878c1404aa82e34371
> |
```

Client2: matching epoch 0 and epoch 1 fingerprints.

```
jay@server: ~/ X jay@client1: ~ X jay@client2: ~ X jay@mallory: ~ X Windows Pow X + - □ X
jay@client1:~/phase5$ ./client 192.168.56.10 5000 alice
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: 1dd6d72269acc49b36204fc4cc97391b9118b9660dff27f4d78df6
d476c624e7
Client-server fingerprint: 1dd6d72269acc49b36204fc4cc97391b9118b9660dff27f4d78df6d476c624e7
Connected as 'alice'. Commands: @user msg | /chat user | /e2e user | /rekey user | /who | /quit
(/rekey forces an immediate rotation for testing, instead of waiting 60s -- only works if your username
sorts before your peer's, since that side is the designated initiator.)
> /e2e
No chat partner selected.
> /e2e bob
[E2E] Initiated key exchange with bob
>
[13:23:18] [E2E] Session with bob now at epoch 0. Fingerprint: 73bbde310dbe964014783a0e325e594b118114ec
811c01ee62b3142852a13045
> /rekey bob
[REKEY] Forced immediate rotation to epoch 1 with bob (test command)
>
[13:23:28] [E2E] Session with bob now at epoch 1. Fingerprint: 25baaa9cbc618e6e97daea5abb7a72e30d9302ac
dde221878c1404aa82e34371
> /rekey bob
[REKEY] Forced immediate rotation to epoch 2 with bob (test command)
>
[13:23:57] [E2E] Session with bob now at epoch 2. Fingerprint: 2b9a88cea1f83815e89b6ee0127919dc72ea92a6
f9ca55cfa10ddb623d85422a
>
```

Client1: continuing to epoch 2 — a third, different fingerprint, timestamped.

```
jay@server: ~/ X jay@client1: ~ X jay@client2: ~ X jay@mallory: ~ X Windows Pow X + - □ X
jay@client2:~/phase5$ ./client 192.168.56.10 5000 bob
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: a56c9ca9bdd8e22dc2a028af7e693c81c5ce4b2171db62a77f928d
55f59ae69e
Client-server fingerprint: a56c9ca9bdd8e22dc2a028af7e693c81c5ce4b2171db62a77f928d55f59ae69e
Connected as 'bob'. Commands: @user msg | /chat user | /e2e user | /rekey user | /who | /quit
(/rekey forces an immediate rotation for testing, instead of waiting 60s -- only works if your username
sorts before your peer's, since that side is the designated initiator.)
>
[13:23:18] [E2E] Session with alice now at epoch 0. Fingerprint: 73bbde310dbe964014783a0e325e594b118114
ec811c01ee62b3142852a13045
>
[13:23:28] [E2E] Session with alice now at epoch 1. Fingerprint: 25baaa9cbc618e6e97daea5abb7a72e30d9302
acdde221878c1404aa82e34371
>
[13:23:57] [E2E] Session with alice now at epoch 2. Fingerprint: 2b9a88cea1f83815e89b6ee0127919dc72ea92
a6f9ca55cfa10ddb623d85422a
> |
```

Client2: matching epoch 2 fingerprint.

Post-rotation message delivery

A message sent immediately after a rotation completed was correctly decrypted by the recipient, displayed with its epoch tag, confirming the rotation did not disrupt the live session.

```
jay@server: ~/ X jay@client1: ~ X jay@client2: ~ X jay@mallory: ~ X Windows Pow X + - □ X
jay@client1:~/phase5$ ./client 192.168.56.10 5000 alice
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: 1dd6d72269acc49b36204fc4cc97391b9118b9660dff27f4d78df6d476c624e7
Client-server fingerprint: 1dd6d72269acc49b36204fc4cc97391b9118b9660dff27f4d78df6d476c624e7
Connected as 'alice'. Commands: @user msg | /chat user | /e2e user | /rekey user | /who | /quit
(/rekey forces an immediate rotation for testing, instead of waiting 60s -- only works if your username
sorts before your peer's, since that side is the designated initiator.)
> /e2e
No chat partner selected.
> /e2e bob
[E2E] Initiated key exchange with bob
>
[13:23:18] [E2E] Session with bob now at epoch 0. Fingerprint: 73bbde310dbe964014783a0e325e594b118114ec811c01ee62b3142852a13045
> /rekey bob
[REKEY] Forced immediate rotation to epoch 1 with bob (test command)
>
[13:23:28] [E2E] Session with bob now at epoch 1. Fingerprint: 25baaa9cbc618e6e97daea5abb7a72e30d9302acdde221878c1404aa82e34371
> /rekey bob
[REKEY] Forced immediate rotation to epoch 2 with bob (test command)
>
[13:23:57] [E2E] Session with bob now at epoch 2. Fingerprint: 2b9a88cea1f83815e89b6ee0127919dc72ea92a6f9ca55cfa10ddb623d85422a
>
[ bob (E2E, epoch 2)]: message sent after rotation
> |
```

Client1: message sent after rotation, tagged with the current epoch, delivered correctly.

```
jay@server: ~/ X jay@client1: ~ X jay@client2: ~ X jay@mallory: ~ X Windows Pow X + - □ X
jay@client2:~/phase5$ ./client 192.168.56.10 5000 bob
Certificate validated: CN=chatserver.local
Proof-of-possession verified.
[client] DH handshake complete. Key fingerprint: a56c9ca9bdd8e22dc2a028af7e693c81c5ce4b2171db62a77f928d55f59ae69e
Client-server fingerprint: a56c9ca9bdd8e22dc2a028af7e693c81c5ce4b2171db62a77f928d55f59ae69e
Connected as 'bob'. Commands: @user msg | /chat user | /e2e user | /rekey user | /who | /quit
(/rekey forces an immediate rotation for testing, instead of waiting 60s -- only works if your username
sorts before your peer's, since that side is the designated initiator.)
>
[13:23:18] [E2E] Session with alice now at epoch 0. Fingerprint: 73bbde310dbe964014783a0e325e594b118114ec811c01ee62b3142852a13045
>
[13:23:28] [E2E] Session with alice now at epoch 1. Fingerprint: 25baaa9cbc618e6e97daea5abb7a72e30d9302acdde221878c1404aa82e34371
>
[13:23:57] [E2E] Session with alice now at epoch 2. Fingerprint: 2b9a88cea1f83815e89b6ee0127919dc72ea92a6f9ca55cfa10ddb623d85422a
> @alice message sent after rotation
> |
```

Client2: correctly received and decrypted the post-rotation message.

Written Explanation (spec §6.2, explicitly graded)

What forward secrecy actually buys you

In Phase 4 alone, C1 and C2 use one static E2E session key for the entire conversation. If an attacker later compromises that single key — say, by seizing a device or exploiting a bug — and had been logging the encrypted traffic throughout, they can decrypt every message from the entire session, past and future, since it was all protected by the same unchanging key.

With Phase 5's rotation, each ~60-second window uses a fresh key derived from a brand-new, independent DH exchange (not derived from the old key). Once a window ends, that window's key is actively erased from memory.

Concretely: if an attacker compromises the key active during, say, minute 5 of a conversation, they can decrypt only the traffic sent during that ~60-second window. They cannot decrypt earlier traffic (minutes 0–4), because those windows' keys were independently derived and have already been destroyed — there is no mathematical path from the minute-5 key back to an earlier one. They also cannot decrypt later traffic (minute 6 onward) without separately compromising each subsequent key, since future keys come from fresh DH exchanges the attacker was not part of.

What forward secrecy does not protect against: an attacker with ongoing, continuous access to a compromised endpoint (e.g., malware actively running in memory during every rotation) could still capture each new key as it is generated. Forward secrecy specifically limits the blast radius of a single, later, point-in-time compromise of previously-discarded key material — it does not make an actively-controlled endpoint secure.

This is the concrete contrast with Phase 4: one static key there means one compromise exposes the whole conversation. Here, one compromise exposes only one ~60-second window, and the size of that window is a direct, tunable security/latency tradeoff.

Appendix A — Generative AI Usage

Prompt 1: "How do I implement a non-blocking line reader in C++ using raw sockets for a TCP chat server?"

Prompt 2: "What is the correct OpenSSL EVP API sequence for AES-256-GCM encryption and decryption in C++, including authentication tag verification?"

Prompt 3: "How can I use OpenSSL BN functions to perform modular exponentiation for a custom Diffie-Hellman implementation without using deprecated DH APIs?"

Prompt 4: "What are the exact X509 API functions in OpenSSL to verify a certificate signature against a trusted CA certificate and check its validity dates?"

Prompt 5: "How do I implement automated 60-second key rotation with epoch tagging for end-to-end encrypted chat sessions in C++?"

Prompt 6: "What are the correct syntax and best practices for utilizing `OPENSSL_cleanse()` to safely zero out expired session keys from memory during rekeying events?"

Prompt 7: "How can I structure a netplan configuration file across multiple Ubuntu 24.04 virtual machines to maintain a static isolated host-only network without DHCP conflicts?"